DepChain: A Dependable Permissioned Blockchain System

Rodrigo Perestrelo (ist1106074), Francisco Heleno (ist1106970), and Eduardo Silva (ist1106929)

Group 1
Instituto Superior Técnico, Universidade de Lisboa

Abstract. This report details the architecture and implementation of the DepChain project, a simplified permissioned blockchain system built on the Basic HotStuff consensus protocol. The system tolerates up to f Byzantine faults among $n = 3f+1$ blockchain members by combining threshold signatures for quorum certification, authenticated perfect links for secure communication, and a multi-layered defence-in-depth strategy that gates every transition from client request to committed state. We present an explicit analysis of the threat model, describe the cryptographic and protocol-level countermeasures, and argue the safety and liveness guarantees provided by the system.

Keywords: Blockchain · HotStuff · Byzantine Fault Tolerance · Distributed Systems · Threshold Signatures.

1 Introduction

The DepChain system implements a permissioned, append-only blockchain with high dependability guarantees in a closed, static membership environment. A predefined Public Key Infrastructure (PKI) supplies self-generated ECDSA key pairs and threshold signature key shares that are pre-distributed to every participant before the system starts.

The architecture separates *blockchain members* (servers), which maintain replicated state and run the consensus protocol, from a *client library* that translates application-level `append(string)` calls into signed requests broadcast to all members. The key challenge is bridging the client's simple interface with the BFT consensus primitives (`Init`, `Propose`, `Decide`) while ensuring that no single Byzantine entity — be it a compromised replica, a malicious leader, or a network adversary — can break safety or permanently prevent progress.

Section 2 describes the layered architecture. Section 3 analyses the threat model and the countermeasures. Section 4 argues the dependability guarantees.

2 System Architecture and Design

The system is organized into three strictly layered modules: a network layer, a consensus layer, and a service/client layer.

2.1 Network Layer: Authenticated Perfect Links

Since TLS is not permitted, communication relies on raw UDP (Fair Loss Links). On top of UDP, an *Authenticated Perfect Links* (APL) abstraction provides three properties:

- **APL1 – Reliable delivery.** Every sent message is retransmitted (up to 5 retries, 500 ms apart) until the destination sends back a sequence-numbered ACK.
- **APL2 – No duplication.** A per-sender *sliding window* tracks delivered sequence numbers; duplicates (already-delivered or below the window's lower bound) are silently dropped.
- **APL3 – Authenticity.** Data and ACK packets carry an HMAC-SHA256 tag computed over the packet's type, sequence number, payload, and sender ID. The HMAC key is derived from an ephemeral ECDH key agreement, itself authenticated by ECDSA signatures from the static PKI (see Section 2.1).

Session Establishment. Before any DATA packet is exchanged, each pair of peers performs a mutually authenticated handshake:

1. The initiator generates a fresh EC key pair on the `secp256r1` (NIST P-256) curve and sends the ephemeral public key in a `HANDSHAKE` packet. The packet's MAC field carries an ECDSA signature (SHA256withECDSA) over the raw ephemeral key bytes, produced with the sender's static private key.
2. The receiver looks up the sender's node ID from a static address → ID map, verifies the ECDSA signature against the sender's known public key, stores the ephemeral public key, and replies with its own `HANDSHAKE`.
3. Both sides compute ECDH($myEphPriv, peerEphPub$) and derive the session HMAC key as Base64(SHA-256($sharedSecret$)).

This provides *forward secrecy* (a new ephemeral key pair per session) and *mutual authentication* (ECDSA signatures prevent man-in-the-middle attacks on the ephemeral exchange). If a peer crashes and restarts, it sends a new `HANDSHAKE`; the other side detects the re-initiation, tears down the old session (clearing deduplication windows and sequence counters), and completes a fresh agreement.

2.2 Consensus Layer: Basic HotStuff

The consensus engine implements Algorithm 2 of the Basic HotStuff protocol [1]. Views are numbered sequentially, with round-robin leader election. Each view has four phases (Prepare, Pre-Commit, Commit, Decide) with $O(n)$ message complexity per phase. The implementation uses a lock-free actor model to avoid race conditions during vote aggregation.

Quorum Certificates (QC) & Blocks. A QC proves that at least $2f+1$ replicas voted for a specific block in a given phase and view. It consists of the phase type, view number, block, and an array of $2f+1$ threshold signature shares (Shoup's scheme [2]). Any replica can re-verify a QC using the shared group key. Blocks store a parent hash, a command string, and a pointer to the parent block. Block identity is content-based via SHA-256.

2.3 Client–Service Interaction

The client library broadcasts each JSON-serialized `ClientRequest` (carrying a unique ID and an ECDSA signature) to all n members. Upon receiving a valid request, nodes register the ID in a `knownRequests` map and enqueue the command. After consensus decides the block, every node sends a `CLIENT_RESPONSE` carrying a `CommitProof` — the COMMIT-phase QC. The client verifies this single proof against the shared group key. Since the threshold signature requires $2f+1$ shares, its validity cryptographically proves consensus, eliminating the need to collect $f+1$ individual responses.

3 Threat Analysis and Protection Mechanisms

We consider an adversary that controls up to f of the $n = 3f+1$ blockchain members (Byzantine faults) and may delay, reorder, or drop network messages arbitrarily before GST. The client is assumed to be correct.

3.1 Network-Level

Unreliable channel. UDP provides no delivery guarantees. The APL layer compensates with persistent retransmission (up to 5 retries at 500 ms intervals) and ACKs. Each packet is signed with HMAC-SHA256 using the session key, and the receiver only delivers packets that pass MAC verification. Tampered or replayed packets are silently discarded.

Message injection and spoofing. Without TLS, a network adversary could inject packets. HMAC-SHA256 with the ECDH-derived session key renders such packets unforgeable: the adversary would need to compromise the ephemeral ECDH private key (which never leaves the JVM) or the static ECDSA key. HMAC verification uses constant-time comparison (`MessageDigest.isEqual`) to prevent timing side-channels.

Man-in-the-middle. The ECDH handshake is authenticated: each peer signs its ephemeral public key with its static ECDSA key (`SHA256withECDSA` on `secp256r1`). The receiver verifies the signature against the sender's known public key (from the pre-distributed PKI). Any MITM interception fails because the attacker cannot produce a valid ECDSA signature for a forged ephemeral key.

3.2 Byzantine Leader

QC forgery. A malicious leader could attempt to fabricate a Quorum Certificate without collecting honest votes. This is prevented by threshold signatures: forming a valid QC requires $k = 2f+1$ threshold signature shares, and the adversary controls at most f key shares. Every replica asynchronously verifies the QC's threshold signature before updating its state or voting.

Fabricated commands. A Byzantine leader might propose a command that no client ever requested. To prevent this, every node independently verifies the client's ECDSA signature upon receiving a `CLIENT_REQUEST` and registers the request ID in a `knownRequests` concurrent map. When a replica receives a `PREPARE` message, it checks `isKnownCommand`: if the command is absent from its

registry, the replica refuses to vote. Without $2f+1$ votes, the forged proposal cannot form a `prepareQC`, and the view times out.

Equivocation (conflicting proposals). A Byzantine leader might send different blocks to different replicas in the same view. The `safeNode` predicate protects against this: a replica votes for block b only if (1) b extends the block certified by its `lockedQC`, or (2) the justifying QC has a higher view number than `lockedQC`. After a replica locks on a block in the COMMIT phase (updating `lockedQC`), it will reject any conflicting block whose justification is weaker. Combined with the quorum intersection property ($|Q_1 \cap Q_2| \geq f+1$ for any two quorums of size $2f+1$), this guarantees that two conflicting blocks can never both be committed.

3.3 Byzantine Replica

Vote amplification. A Byzantine replica might send duplicate votes to inflate the vote count. Per-view boolean guards (`sentPrepareSig`, `sentPreCommitSig`, `sentCommitSig`) ensure each correct replica sends at most one vote per phase per view. On the leader side, votes are stored in a map keyed by the signature share ID (1-based node index), so duplicate shares from the same key share overwrite rather than accumulate.

Garbage signatures. A Byzantine replica may send a vote containing corrupted threshold signature bytes. When the leader attempts to form a QC and `tverify` fails, the `onFailure` callback resets the QC-formation guard, allowing the leader to collect replacement votes from honest replicas. The leader is not permanently blocked.

3.4 Client-Side

Request authentication & replay. Client ECDSA signatures are verified twice (defence-in-depth): upon network reception, and before state execution (`UpcallHandler`). Unique IDs prevent replays; duplicates are dropped via an `executedRequestIds` set, and committed IDs are evicted from `knownRequests` to bound memory.

Response verification. Clients do not naively trust individual replicas. They verify the `CommitProof` (a threshold signature over the block hash) against the shared group key, accepting only cryptographically valid proofs of consensus.

4 Dependability Guarantees

4.1 Safety

Agreement. The three-phase locking ensures that if a correct replica commits block b in view v, no correct replica can commit a conflicting block. A block is committed only after forming a *commitQC* ($2f+1$ COMMIT votes). This phase is only reached if $2f+1$ replicas voted PRE-COMMIT, requiring $2f+1$ PREPARE votes. Any two quorums of $2f+1$ out of $3f+1$ overlap in at least $f+1$ nodes (at least one honest node). After voting COMMIT, an honest replica sets *lockedQC* to the PRE-COMMIT QC. The `safeNode` predicate prevents voting for conflicting blocks unless a QC from a strictly higher view is

presented, which is impossible to form without honest votes, contradicting the lock.

Validity & Integrity. Only legitimate client commands (verified ECDSA) are committed. Blocks are hash-chained, making modifications detectable, and the `ServiceState` is strictly append-only.

4.2 Liveness

Liveness (termination) is guaranteed after the Global Stabilization Time (GST), when network delays become bounded.

View-change mechanism. A view timer fires after a configurable timeout (default 4 s). If the leader is crashed or Byzantine and does not drive consensus to completion, replicas time out and advance to view $v+1$ by sending a `NEW-VIEW` message to the next leader. Exponential backoff (doubling the timeout after each failed active view, capped at 32 s) prevents livelock from repeated view changes during transient network instability. When a view completes successfully, the timeout resets to the default, keeping latency low under normal conditions.

Idle views (no pending command and no proposal) do not trigger backoff — the timeout resets on entry, ensuring the system responds quickly when a new client command eventually arrives.

Leader rotation. Round-robin leader election ($v \bmod n$) ensures that every non-faulty replica eventually becomes the leader. Since at most f are Byzantine, a correct leader is guaranteed within at most $f+1$ consecutive view changes.

Quorum availability. With $n = 3f+1$ replicas and at most f faulty, there are always at least $2f+1$ correct replicas — exactly the quorum threshold.

Safe catch-up. If a replica receives a QC from a future view (e.g. after recovering from a temporary network partition), it asynchronously verifies the QC and, if valid, jumps to the corresponding view. This prevents slow replicas from falling permanently behind, which would reduce the effective quorum size.

5 Conclusion

DepChain demonstrates that the combination of the Basic HotStuff consensus protocol, threshold signature-based quorum certificates, and authenticated perfect links over UDP provides a robust foundation for a Byzantine fault-tolerant blockchain. The multi-layered defence-in-depth approach — spanning ECDH session establishment, HMAC-authenticated packets, ECDSA-signed client requests, threshold-verified quorum certificates, and protocol-level safety predicates — ensures that no single point of failure can compromise the system's safety, while view-change timeouts with exponential backoff guarantee eventual progress.

References

1. Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., Abraham, I.: HotStuff: BFT Consensus in the Lens of Blockchain. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC '19), pp. 347–356. ACM (2019).
2. Shoup, V.: Practical Threshold Signatures. In: Advances in Cryptology — EUROCRYPT 2000, LNCS, vol. 1807, pp. 207–220. Springer (2000).